

TP previo Parcial 2	DDS 2026 - Curso 3k4
Materia	Desarrollo de Software (DDS)
Año	2026
Instancia de evaluación	TP previo - Parcial 2
Documento	Enunciado de trabajo práctico
Curso	3K4

Introducción

Desde la publicación de la tarea hasta el sábado 13/06 a las 23:55 hs. cada grupo de entre 2 y 5 alumnos desarrollará una aplicación full stack para seguimiento de tareas dentro de proyectos. Adicionalmente cada alumno deberá subir el trabajo realizado a su portafolio.

Desde el 14/6 10 hs hasta el lunes 15/6 a las 23:55 otro grupo deberá realizar una revisión / corrección entre pares (revisión entre grupos) y entregar la devolución grupal.

Estas dos tareas son esenciales y previas al segundo parcial ya que habrá 2 preguntas a desarrollar relativas el tema desarrollado.

El problema exige coordinar proyectos, responsables, prioridades, vencimientos, estados y permisos, como ocurriría en una herramienta interna de gestión de trabajo. El objetivo es que la solución permita entender el avance real de los proyectos y no solo crear tareas sueltas.

Este práctico integra los contenidos principales del segundo parcial: Express Router, middlewares, manejo centralizado de errores, CORS, autenticación y autorización con JWT, pruebas de API con Jest y Supertest, React con Vite, componentes, estado, efectos, React Router, formularios, Axios, validaciones y manejo de errores de API.

Como el trabajo se resuelve en grupo y con varios días de desarrollo, se espera una solución de complejidad media-alta: reglas de negocio reales, estados, permisos, historial, filtros avanzados, persistencia, datos semilla, pruebas automatizadas y una interfaz navegable que permita operar el sistema completo.

Alcance mínimo de resolución grupal

La entrega debe evidenciar que hubo división de responsabilidades y posterior integración. Como mínimo, el sistema debe incluir:

- Un módulo de autenticación completo, con registro, login, JWT, roles y persistencia de sesión en frontend;
- Un módulo principal del dominio con listado, filtros, detalle, alta, edición y acciones de estado;
- Un módulo de administración o resumen que muestre información agregada del dominio;
- Válidaciones de negocio implementadas en servicios del backend, no solamente en formularios;
- Al menos dos flujos end-to-end funcionando desde la interfaz hasta la persistencia;

- Pruebas automatizadas que cubran casos exitosos, errores de validación, errores de permisos y
- Reglas específicas del dominio;
- documentación suficiente para que otro grupo pueda ejecutar, probar y validar la solución.

No se considera suficiente una entrega que solo tenga pantallas sin backend real, endpoints sin frontend integrado, datos hardcodeados en componentes, o reglas de negocio aplicadas únicamente de forma visual.

¿Qué deben construir?

Desarrollar una aplicación web full stack para seguimiento de tareas de proyectos. La aplicación debe permitir crear tareas dentro de proyectos, asignarlas a responsables válidos y controlar prioridad y estado. No es un CRUD genérico: una tarea debe pertenecer a un proyecto, tener un responsable permitido y respetar transiciones de estado coherentes.

En el parcial presencial cada estudiante responderá preguntas escritas sobre la solución entregada por su grupo. Deben poder explicar rutas, componentes, servicios, middlewares, validaciones, JWT, permisos, pruebas y errores manejados.

Dominio obligatorio

El sistema trabaja sobre el dominio de seguimiento de proyectos. El recurso principal que se administra es la Tarea: una unidad de trabajo con responsable, prioridad, estado y fecha límite. Cada tarea debe estar asociada obligatoriamente a un Proyecto, que agrupa integrantes, estado general y contexto de trabajo.

La regla central del dominio es que una tarea solo puede crearse o reasignarse si el proyecto existe, no está finalizado, el responsable pertenece al proyecto y la prioridad y el estado pertenecen a los valores permitidos. Los cambios de estado deben representar un avance coherente del trabajo.

Entidades mínimas obligatorias

El sistema debe modelar y persistir al menos 4 entidades. Si usan archivo JSON, SQLite, Sequelize u otra alternativa, igual deben respetar esta separación lógica:

Entidad	Para qué existe	Campos mínimos	Relaciones principales
usuarios	Representa colaboradores, líderes y administradores que trabajan sobre proyectos.	id, nombre, email, passwordHash, rol, activo	Un usuario puede integrar proyectos, ser responsable de tareas y generar historial.
proyectos	Representa el contenedor de trabajo donde se organizan tareas.	id, codigo, nombre, descripcion, estado, integrantes	Un proyecto tiene muchas tareas y varios usuarios integrantes.
tareas	Representa una unidad de trabajo con responsable, prioridad y estado.	id, proyectoId, titulo, descripcion, responsableId, prioridad, estado, fechaLimite, createdAt	Pertenece a un proyecto y tiene un usuario responsable.
historial_tareas	Registra auditoría de cambios sobre tareas.	id, tareaId, usuarioId, accion, fechaHora, valorAnterior, valorNuevo	Pertenece a una tarea y al usuario que hizo la acción.

Campos minimos por entidad

Los campos pueden adaptarse al motor de persistencia elegido, pero la informacion y las relaciones deben estar presentes.

Usuarios

Campo	Tipo	sugerido	Obligatorio	Descripción	Ejemplo o valores posibles
id	numero/string		Sí	Identificador único del usuario.	1, usr-001
nombre	string		Sí	Nombre visible del usuario.	Mica Torres
email	string		Sí	Credencial de acceso y dato de contacto. Debe ser único.	mica@dds.com
passwordHash	string		Sí	Contraseña almacenada de forma segura o hash simulado documentado.	bcrypt_hash
rol	string		Sí	Define permisos dentro del sistema.	colaborador, lider, admin

Campo	Tipo	sugerido	Obligatorio	Descripción	Ejemplo o valores posibles
activo	boolean	Sí		Permite bloquear accesos sin borrar el usuario.	true, false

Proyectos

Campo	Tipo	sugerido	Obligatorio	Descripción	Ejemplo o valores posibles
id	numero/string	Sí		Identificador único del proyecto.	proy-001
codigo	string	Sí		Código corto para identificar el proyecto. Debe ser único.	DDS-API
nombre	string	Sí		Nombre visible del proyecto.	Portal de alumnos
descripcion	string	Sí		Objetivo o alcance del proyecto.	Sistema interno de seguimiento académico
estado	string	Sí		Estado general del proyecto.	activo, pausado, finalizado
integrantes	array	Sí		Lista de usuarios habilitados para recibir tareas.	["usr-001", "usr-002"]

Tareas

Campo	Tipo	sugerido	Obligatorio	Descripción	Ejemplo o valores posibles
id	numero/string	Sí		Identificador único de la tarea.	tar-1001
proyectoId	numero/string	Sí		Proyecto al que pertenece.	proy-001

Campo	Tipo sugerido	Obligatorio	Descripción	Ejemplo o valores posibles
			Debe existir en proyectos.	
titulo	string	Sí	Resumen corto de la tarea.	Implementar login
descripcion	string	Sí	Detalle del trabajo esperado.	Crear endpoint y pantalla de login
responsableId	numero/string	Sí	Usuario asignado. Debe integrar el proyecto.	usr-001
prioridad	string	Sí	Importancia relativa de la tarea.	baja, media, alta, critica
estado	string	Sí	Estado actual del flujo.	pendiente, en_progreso, bloqueada, finalizada, cancelada
fechaLimite	date/string	Sí	Fecha máxima esperada de resolución.	2026-06-21
createdAt	datetime/string	Sí	Fecha y hora de creación.	2026-06-08T10:30:00
HistorialTareas				

Campo	Tipo sugerido	Obligatorio	Descripción	Ejemplo o valores posibles
id	numero/string	Sí	Identificador único del registro de auditoría.	hist-001
tareaId	numero/string	Sí	Tarea afectada por el cambio.	tar-1001
usuarioId	numero/string	Sí	Usuario que realizó la acción.	usr-lider
accion	string	Sí	Operación realizada sobre la tarea.	Creación, edicion, reasignacion, cambio_estado, cancelacion
fechaHora	datetime/string	Sí	Momento exacto de la acción.	2026-06-08T11:15:00
valorAnterior	object/string	No	Datos relevantes antes del cambio.	{"estado": "pendiente"}
valorNuevo	object/string	No	Datos relevantes después del cambio.	{"estado": "en_progreso"}

Reglas funcionales específicas

La aplicación debe resolver estos casos:

1. Registrar usuario e iniciar sesión.
2. Obtener un JWT al iniciar sesión correctamente.
3. Listar tareas con filtros combinables por proyectoId, responsableId, estado y prioridad.
4. Ver el detalle de una tarea desde una ruta dinámica.
5. Crear una tarea dentro de un proyecto existente.
6. Editar título, descripción, prioridad, responsable, fecha límite o estado segun permisos.
7. Cancelar una tarea cambiando estado a cancelada.
8. Rechazar tareas con proyecto inexistente o proyecto finalizado.

9. Rechazar tareas cuyo responsableId no figure en integrantes del proyecto.
10. Rechazar prioridades y estados que no pertenezcan a los valores permitidos.

Complejidad adicional obligatoria

El grupo debe implementar también:

1. Flujo completo de estados: pendiente -> en_progreso ; en_progreso -> bloqueada o finalizada; cualquier tarea no finalizada -> cancelada. No permitir editar tareas finalizada o cancelada, salvo observaciones administrativas.
2. búsqueda paginada y ordenable: aceptar page, limit, sortBy y order en el listado de tareas.
3. Vista resumen para administración con tareas por estado, tareas vencidas, carga por responsable y cantidad de tareas críticas.
4. Historial de cambios de cada tarea: guardar fecha, usuario, acción y valores modificados cuando se reasigna, cambia prioridad, cambia estado o se edita.
5. Cálculo de tareas vencidas cuando fechaLimite sea anterior a la fecha actual y el estado no sea finalizada ni cancelada.
6. validación de que un proyecto pausado solo permita editar tareas existentes, pero no crear nuevas.
7. Semilla de datos inicial con al menos 4 proyectos, 5 usuarios, 1 lider/admin y 15 tareas en distintos estados.
8. Mensajes de error distintos para proyecto inexistente, responsable inválido, estado inválido, prioridad inválida y falta de permisos.

Aclaraciones para resolver el alcance

Estas aclaraciones responden dudas esperables al momento de implementar:

¿Quién puede ser responsable de una tarea? Solo un usuario incluido en integrantes del proyecto asociado.

¿Qué pasa si el proyecto esta pausado? No se pueden crear tareas nuevas, pero si se pueden consultar y editar tareas existentes según permisos.

¿Qué pasa si el proyecto está finalizado? No se pueden crear tareas nuevas ni modificar tareas, salvo que el grupo documente una acción administrativa muy puntual.

¿Qué tareas se consideran vencidas? Las que tienen fechaLimite anterior a la fecha actual y no están finalizada ni cancelada.

¿Qué debe mostrar el resumen? Como mínimo: tareas por estado, tareas vencidas, tareas por responsable y tareas de prioridad crítica.

¿Dónde se validan responsables, estados y prioridades? En el servicio del backend. El frontend puede ofrecer selects, pero no alcanza como validación.

Roles y permisos

Deben existir al menos estos roles:

-colaborador: puede ver tareas asignadas, actualizar descripción o pasar sus tareas a en_progreso o bloqueada.

-admin o líder: puede crear tareas, reasignarlas, cambiar prioridad, finalizar o cancelar cualquier tarea del proyecto.

Al menos una ruta de escritura debe devolver:

401 si no se envia JWT.

403 si el usuario está autenticado pero no tiene permiso para la acción.

Backend esperado

Implementar backend con Node.js y Express.

Rutas mínimas:

```
POST /api/auth/register
POST /api/auth/login
GET /api/proyectos
GET /api/tareas?proyectoId=&responsableId=&estado=&prioridad=
GET /api/tareas/resumen
GET /api/tareas/:id

GET /api/tareas/:id/historial
POST /api/tareas
PUT /api/tareas/:id
PATCH /api/tareas/:id/iniciar
PATCH /api/tareas/:id/bloquear
PATCH /api/tareas/:id/cancelar
PATCH /api/tareas/:id/finalizar
```

Estructura mínima:

- routes/tareas.routes.js con express.Router()
- controlador de tareas
- servicio de tareas con reglas de responsable, prioridad y estado
- middleware de autenticación JWT
- middleware de autorización por rol o responsable
- middleware de validación de entrada
- middleware de manejo de errores con firma (err, req, res, next)

- persistencia en archivo JSON, SQLite, Sequelize u otra solución que conserve datos al reiniciar el backend
- carga de datos semilla para probar el dominio sin cargar todo manualmente

Usar status HTTP coherentes: 200, 201, 400, 401, 403, 404 y 500. Las respuestas de error deben tener JSON claro, por ejemplo {"error": "El responsable no pertenece al proyecto"}.

Frontend esperado

Implementar frontend con React y Vite.

Pantallas minimas:

1. Login y registro.
2. Listado de tareas con filtros por proyecto, responsable, estado y prioridad.
3. Detalle de tarea en una ruta como /tareas/:id .
4. Pantalla transaccional de alta/edición de tarea, donde se selecciona proyecto, responsable, prioridad, fecha límite y estado inicial, y se confirma la operación contra la API.
5. Acciones visibles para cambiar estado, finalizar o cancelar según rol.
6. Panel resumen para administradores o lideres.
7. Historial visible dentro del detalle de una tarea.
8. Ruta comodín para página no encontrada.

La capa de servicios debe usar Axios:

1. instancia con baseUrl
2. params para filtros
3. body en POST, PUT o PATCH
4. header Authorization: Bearer <token> en acciones protegidas
5. manejo visible de errores de validacion, autenticacion, autorizacion y recurso inexistente
6. componentes separados para tabla/listado, filtros, formulario, detalle, acciones por rol y resumen administrativo
7. estados de carga, vacío, error y éxito para las operaciones principales

integración y calidad esperada

La resolución debe mostrar trabajo de equipo y no una acumulación de archivos sueltos.

Se espera:

- contexto, hook o mecanismo equivalente para conservar usuario autenticado, token y rol en frontend;
- rutas protegidas en frontend y backend, no solo botones ocultos;
- validaciones repetidas en frontend para experiencia de usuario y en backend como fuente de verdad;
- servicios Axios separados por recurso, sin llamadas HTTP mezcladas dentro de todos los componentes;
- contraseñas hasheadas o, si usan usuarios semilla simplificados, aclaración explícita de la limitación en el README;

- payload del JWT sin contraseñas ni datos sensibles;
- manejo de errores centralizado en backend y mensajes comprensibles en frontend;
- decisión documentada sobre persistencia, estructura de carpetas y división de responsabilidades del grupo.

Testing mínimo

El backend debe incluir pruebas con Jest y Supertest para:

1. Login correcto e invalido.
2. Listado de tareas con y sin filtros.
3. Detalle de tarea existente e inexistente.
4. Creación válida de una tarea.
5. Creación inválida por responsable fuera del proyecto.
6. Creación inválida por prioridad o estado no permitido.
7. Acceso sin JWT a una ruta protegida.
8. Acceso con JWT de colaborador a una acción solo permitida para admin o lider.
9. Creación inválida sobre proyecto finalizado o pausado.
10. Transición de estado no permitida, por ejemplo, volver una tarea finalizada a pendiente.

Las pruebas deben verificar status HTTP y cuerpo JSON relevante.

documentación obligatoria

La entrega debe incluir un README.md con:

- instrucciones para ejecutar backend y frontend
- usuario admin o lider y usuario colaborador de prueba
- listado de endpoints principales
- listado de rutas frontend
- explicación de responsable valido, prioridad y estados
- explicación de JWT, roles y permisos
- comando para ejecutar pruebas
- limitaciones conocidas

Preguntas frecuentes y criterios de corrección

Sobre alcance y evaluación

El TP es condición de entrega y también será la base de preguntas abiertas del Parcial 2. La nota del parcial surge del examen, pero una entrega incompleta o no explicable afecta directamente las respuestas escritas.

En el parcial se preguntará sobre el código real entregado por el grupo: rutas, servicios, componentes, validaciones, JWT, permisos, pruebas y decisiones.

Se aceptan soluciones equivalentes si respetan entidades mínimas, reglas del dominio, rutas, pantallas, pruebas y criterios de seguridad. Cualquier cambio importante debe estar justificado en el README.

La persistencia puede ser archivo JSON, SQLite, Sequelize u otra alternativa, siempre que conserve datos al reiniciar y respete las entidades del enunciado.

Debe haber registro/login real desde frontend. Los usuarios semilla son obligatorios para probar rápido, pero no reemplazan el flujo de autenticación.

El historial debe existir en backend y verse desde frontend, al menos en el detalle del recurso principal.

La pantalla transaccional puede ser una sola pantalla para alta/edición o dos pantallas separadas, pero debe confirmar la operación contra la API y mostrar errores.

El resumen administrativo debe estar protegido para roles administrativos.

Filtros, paginación y ordenamiento deben resolverse en backend. El frontend solo envía params y muestra resultados.

Documentar una limitación no reemplaza un requisito mínimo. Sirve para explicar decisiones o faltantes menores, no para omitir reglas centrales.

Sobre backend

Deben usar `express.Router()` en archivos separados para el recurso principal.

El middleware de errores con firma (`err, req, res, next`) debe quedar después de las rutas.

Los errores de validación deben responder con status coherente, normalmente 400, y JSON claro, por ejemplo `{"error": "mensaje"}`.

La autorización debe validar rol y, cuando corresponda, propiedad del recurso. No alcanza con que el JWT exista.

El JWT no debe incluir contraseñas ni datos sensibles.

Las contraseñas deben guardarse hasheadas; si se usa una simplificación por tiempo, debe estar documentada y no exponer contraseñas en respuestas.

Las reglas de negocio deben estar en servicios del backend, no solamente en controladores ni formularios.

Deben estar protegidas las acciones de escritura y cualquier lectura privada o administrativa.

Sobre frontend

React Router debe incluir ruta comodin *.

Las pantallas de detalle deben leer parametros con useParams .

Pueden usar React Hook Form o formularios controlados con useState, pero las validaciones deben verse en pantalla.

Los errores de API deben mostrarse de forma comprensible cerca de la acción que falló o en un área visible de la pantalla.

Axios debe estar en una capa de servicios separada. No mezclar llamadas HTTP en todos los componentes.

El JWT puede enviarse con interceptores o agregando headers en servicios, pero siempre como `Authorization: Bearer <token>` .

Proteger una ruta en frontend significa impedir acceso visual/navegable si no hay usuario autenticado o rol suficiente; esto no reemplaza la protección del backend.

Sobre testing

Se exigen pruebas de backend con Jest y Supertest. Tests de frontend son opcionales.

Cada test debe validar status HTTP y cuerpo JSON relevante.

Deben probar login correcto, login inválido, acceso sin token y acceso con rol insuficiente.

Deben probar reglas específicas del dominio, no solo endpoints felices.

Los datos de prueba deben ser previsibles. Si un test modifica datos, deben resetearlos o aislarlos para no afectar otros tests.

Entrega

Subir a Moodle el repositorio o archivo comprimido en el portafolio del alumno y en la entrega grupal.

No incluir carpetas `node_modules`

Si deben incluir `package.json` , código fuente, tests y documentación.